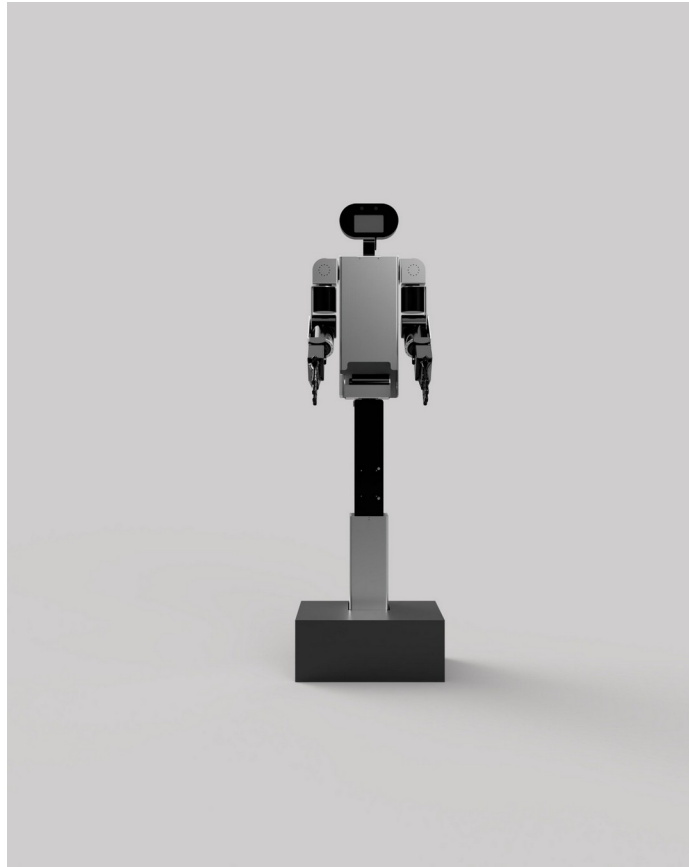


Buildo Developer Platform

MuJoCo Simulation, VLA Post-Training, App Store Publishing and Real-Robot Deployment



Buildo concept: OpenArm-style dual-arm upper body on a mobile, height-adjustable base.

Platform principle

Buildo should behave like a robot application platform: developers collect demonstrations, post-train task intelligence, validate it in MuJoCo, publish a signed robot app, and deploy the same app contract to simulated and real Buildo robots.

End-to-end flow



1. Update the OpenArm MuJoCo Repository for Buildo

Use the existing OpenArm MuJoCo model as the arm subsystem and build Buildo around it. Avoid rewriting the OpenArm model unless a Buildo-specific mechanical change requires it.

- Fork or wrap the OpenArm MuJoCo repository and keep the upstream OpenArm model as a clean dependency.
- Add the Buildo mobile base, height-adjustable column, head/cameras and Buildo hands as separate MJCF files.
- Keep the first mobile-base model simple: x/y translation, yaw rotation and a vertical lift joint.
- After the physical wheel design is frozen, replace the simplified planar base with the actual wheel joints, masses, friction and drive kinematics.
- Keep VLA actions in task space. Local controllers should translate high-level base velocity, lift height, wrist pose and hand targets into joint commands.

Suggested repository structure

```
buildo_mujoco/  
|  
|-- robots/  
|   |-- openarm/  
|   |-- buildo/  
|       |-- buildo.xml  
|       |-- mobile_base.xml  
|       |-- lifter.xml  
|       |-- head.xml  
|       |-- hands.xml  
|  
|-- scenes/  
|   |-- home/  
|   |-- office/  
|   |-- warehouse/  
|  
|-- objects/  
|   |-- coffee_machine/  
|   |-- refrigerator/  
|   |-- washing_machine/  
|   |-- furniture/  
|  
|-- controllers/  
|-- environments/  
|-- policy_client/  
|-- app_runtime/  
|-- tests/
```

Recommended VLA-facing action space

Subsystem	VLA command
Mobile base	vx, vy, yaw rate
Height mechanism	Lift height or lift velocity
Left arm	End-effector pose / trajectory
Right arm	End-effector pose / trajectory
Left hand	Hand / finger target
Right hand	Hand / finger target

2. Add Home, Office and Warehouse Environments

Keep environments independent from the robot model so the same Buildo robot can be dropped into many scenes without changing its MJCF.



Concept view of a Buildo home simulation environment with task zones, object recognition, navigation paths, manipulation targets and digital-twin validation.

- Create separate scene files such as home_01.xml, office_01.xml and warehouse_01.xml.
- Use detailed visual meshes but simple collision geometry to keep simulation fast.
- Organize each scene into functional zones such as kitchen, living room, laundry, work area or storage.
- Add environment randomization from the beginning so developers can test robustness across layouts.

Example home scene

```

Home
|-- Kitchen
|   |-- counter
|   |-- sink
|   |-- refrigerator
|   |-- coffee machine
|   |-- cabinets
|
|-- Living room
|   |-- sofa
|   |-- table
|   |-- chairs
|
|-- Laundry
|   |-- washing machine
|   |-- laundry basket
|   |-- shelves

```

Useful randomization

Parameter	Example range / variation
-----------	---------------------------

Coffee-machine position	+/- 10 cm
Counter height	+/- 5 cm
Lighting	Multiple intensities and directions
Objects	Different cups, bottles, boxes
Appliances	Multiple visual and control variants
Furniture	Small position and orientation changes

3. Add Interactive Appliances and Objects

A useful simulation asset is more than a 3D mesh. Each appliance should include interaction points, joints, state and simple task logic.

- Coffee machine: power button, brew button, cup area, dispenser location and a simple brew state machine.
- Refrigerator: hinged door, handle interaction site, internal target objects and door-angle state.
- Washing machine: door hinge, drum, start button, program dial and clothes objects.
- Attach a small manifest to every interactive asset so task compatibility can later be checked automatically.

Example appliance manifest

```
asset_id: coffee_machine_v1
type: coffee_machine
```

```
affordances:
- press_power
- place_cup
- press_brew
```

```
interaction_sites:
power_button: [x, y, z]
brew_button: [x, y, z]
cup_area: [x, y, z]
```

```
dimensions:
width: ...
height: ...
depth: ...
```

4. Buildo Studio: Import Data and Post-Train the VLA

Buildo Studio should make robot app development feel like an application workflow, not a collection of ML scripts.

```
+-----+
| BUILD0 STUDIO |
+-----+
| Data | Retarget | Train | Simulator | Publish |
+-----+
```

- Import synchronized sessions exported by the Android glove/glasses app.
- Preview video, glove, tactile and head-motion streams and verify synchronization.
- Retarget human wrist and finger motion into Buildo task-space and hand targets.
- Use simulation to correct failed or unreachable retargeted motions before training.
- Post-train the task/action expert, with optional lightweight VLM adaptation if needed.
- Save a versioned model artifact such as Home-App-v0.1 and bind it to a fixed Buildo observation/action schema.

Example imported session

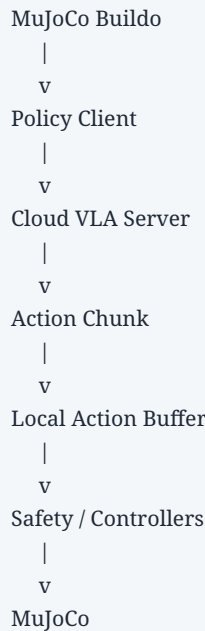
```
session_00042/
|-- video.mp4
|-- head_pose.parquet
|-- left_hand_position.parquet
|-- right_hand_position.parquet
|-- left_glove.parquet
|-- right_glove.parquet
|-- tactile.parquet
|-- timestamps.json
`-- metadata.json
```

Preferred training flow

```
Human gloves + glasses
|
v
Retargeting
|
v
Buildo in MuJoCo
|
v
Developer correction / successful rollout
|
v
Robot-space training trajectory
|
v
Post-train Buildo VLA / action expert
```

5. Connect the Trained VLA Server to Buildo in MuJoCo

Simulation and real-robot deployment should use the same network contract. MuJoCo simply acts as one implementation of the Buildo robot endpoint.



- Send observations at the VLA rate rather than every physics step.
- Return a future action chunk, for example 300-1000 ms of nominal motion.
- Buffer and interpolate the action chunk locally so simulation can keep running between server responses.
- Keep physics, tactile correction and actuator-level safety local.
- Inject latency, packet loss and server pauses during simulation certification.

Example observation/response fields

Observation to server	Action response
Instruction; camera frame(s); base state; lift state; left/right arm state; hand state; timestamp; sequence ID	Model version; action start time; action dt; future base/lift/wrist/hand targets; sequence ID

Test metrics

- Task success rate
- Collision count
- Object drops
- Maximum contact force
- Time to completion
- Inference latency p50 / p95
- Network deadline misses
- Recovery attempts
- Environment configuration and replay video

6. Publish the Tested Model as a Buildo Robot App

A Buildo app should be a signed application package containing capability, compatibility, safety and model information - not only a model checkpoint.

```
home-app/  
|-- manifest.yaml  
|-- tasks.yaml  
|-- compatibility.yaml  
|-- safety.yaml  
|-- model.yaml  
|-- simulator_report.json  
|-- thumbnails/  
`-- developer_signature.sig
```

- Run an automated Buildo Certification Suite before an app can be published.
- Store the verified model version or cloud inference endpoint in the app manifest.
- Expose supported tasks, tested appliance types, tested layouts and workspace requirements to the robot owner.
- Use simulator results to produce a simple home-compatibility score or compatibility checklist.
- Meter inference through the Buildo platform so developer revenue can be tied to actual app usage.

Example Store metadata

Field	Example
App	Home App
Supported tasks	Make coffee; use refrigerator; load laundry; basic cleaning
Robot	Buildo-1
Tested kitchens	23 layouts
Coffee machines	6 variants
Washing machines	4 variants
Runtime	Cloud VLA
Revenue model	Developer earns from app inference usage

7. Connect the Cloud VLA to a Real Buildo over WebRTC

The real robot should use the same BuildoObservation and BuildoActionChunk schema as MuJoCo. Only the robot adapter and network transport change.

```

    CLOUD
  Home App VLA
    ^ |
    | v
  WebRTC
    ^ |
    | v
  BUILD0

```

```

Video -----> media track
Robot state -----> data channel
Action chunks <----- data channel
Control/status <-----> data channel

```

```

    |
    v
  Action Buffer
    |
    v
  Safety Projection
    |
+-----+-----+
|         |         |
|         |         |
v         v         v
tactile controller  base/arms
200-500 Hz         local ctrl
|         |         |
+-----+-----+
|
| v
~1 kHz safety
|
v
Robot

```

- Use a short-lived authenticated session token before opening the WebRTC connection.
- Use media tracks for camera video and DataChannels for robot state, actions and control/status messages.
- Keep 200-500 Hz tactile correction and ~1 kHz actuator safety on the robot.
- The cloud VLA should generate nominal task motion and contact intent at roughly 10-30 Hz.
- If the cloud connection stalls, finish the valid buffered chunk and transition to SAFE HOLD instead of extrapolating indefinitely.
- SAFE HOLD should stop the mobile base, hold the arm target, maintain the grasp locally and keep tactile anti-slip and actuator safety active.

Core architectural decision

Define one stable BuildoObservation and BuildoActionChunk schema.

Use the same schema for retargeting, MuJoCo, cloud model testing, App Store certification and the physical robot. A developer should be able to switch from BuildoSim to BuildoHardware without rewriting the robot app.

References / starting points

OpenArm MuJoCo repository: https://github.com/enactic/openarm_mujoco

OpenArm MuJoCo documentation: <https://docs.openarm.dev/simulation/mujoco/>

OpenArm inference tutorial: <https://docs.openarm.dev/tutorial/inference/>

OpenArm Dora API reference: <https://docs.openarm.dev/api-reference/dora/>

Psi0: <https://psi-lab.ai/Psi0/>

SIMPLE architecture: <https://psi-lab.ai/SIMPLE/docs/core/architecture.html>

SIMPLE replay/render data pipeline: https://psi-lab.ai/SIMPLE/docs/decoupled-wbc/replay_render.html

MuJoCo modeling guide: <https://mujoco.readthedocs.io/en/3.3.0/modeling.html>

WebRTC peer connections: <https://webrtc.org/getting-started/peer-connections>

WebRTC data channels: <https://webrtc.org/getting-started/data-channels>